

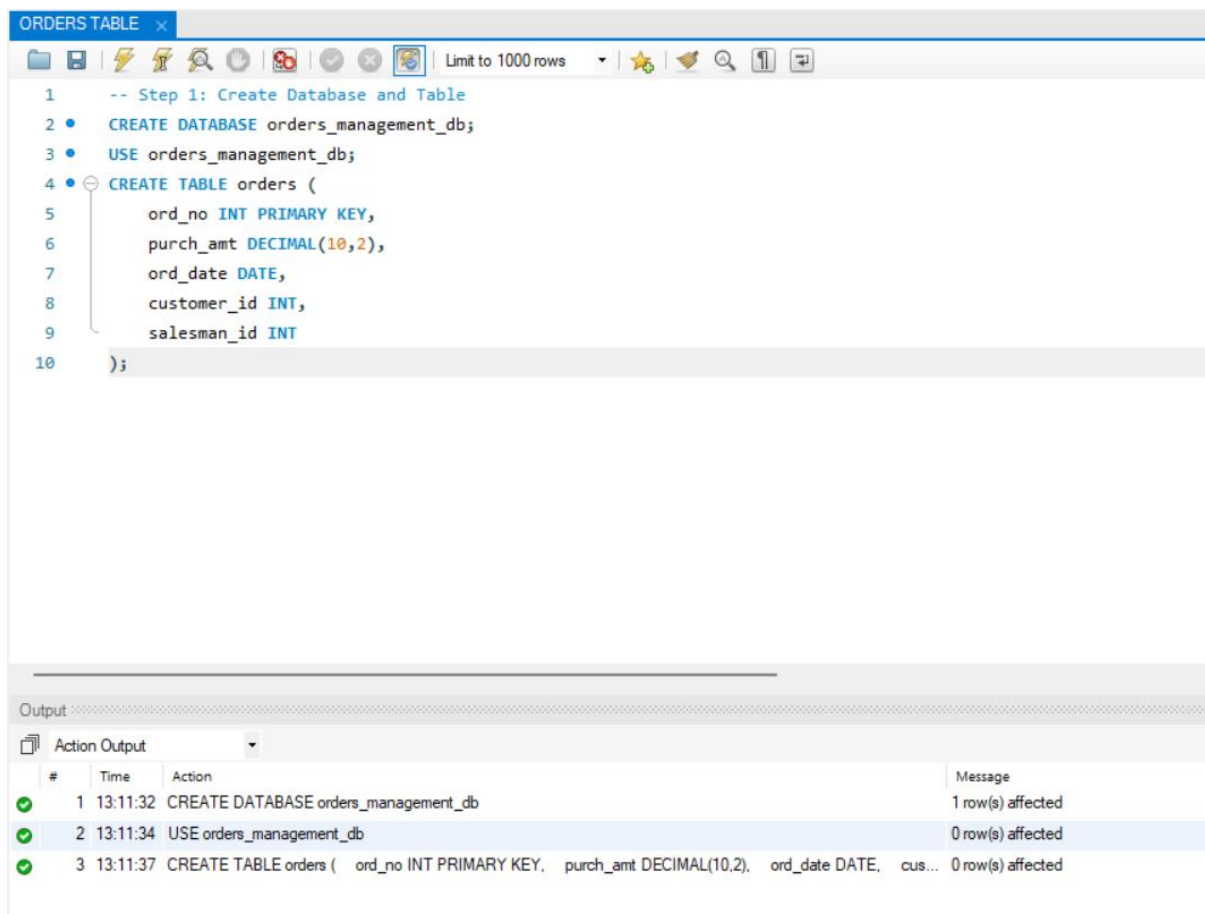
ORDERS TABLE _SQL AGGREGATE & CLAUSE OPERATIONS

Ili.Harshika (2520030607)

Question :

Perform SQL operations on an Orders table using WHERE, ORDER BY, GROUP BY, HAVING, SUM(), AVG(), MAX(), MIN(), COUNT(), and BETWEEN to analyse orders, customer purchases, salesman performance, and purchase amounts.

Step 1 : Create Database and Table



The screenshot displays a SQL IDE interface with a toolbar at the top and a code editor below. The code editor contains the following SQL script:

```
-- Step 1: Create Database and Table
1 CREATE DATABASE orders_management_db;
2 USE orders_management_db;
3 CREATE TABLE orders (
4     ord_no INT PRIMARY KEY,
5     purch_amt DECIMAL(10,2),
6     ord_date DATE,
7     customer_id INT,
8     salesman_id INT
9 );
```

Below the code editor, the 'Output' pane is visible, showing the execution results of the SQL script. The output is as follows:

#	Time	Action	Message
✓ 1	13:11:32	CREATE DATABASE orders_management_db	1 row(s) affected
✓ 2	13:11:34	USE orders_management_db	0 row(s) affected
✓ 3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cus...	0 row(s) affected

ORDERS TABLE

```

6      purch_amt DECIMAL(10,2),
7      ord_date DATE,
8      customer_id INT,
9      salesman_id INT
10     );
11     -- Step 2: Insert Data
12     • INSERT INTO orders VALUES
13     (70001,150.50,'2012-10-05',3005,5002),
14     (70009,270.65,'2012-09-10',3001,5005),
15     (70002,65.26,'2012-10-05',3002,5001),
16     (70004,110.50,'2012-08-17',3009,5003),
17     (70007,948.50,'2012-09-10',3005,5002),
18     (70005,2400.60,'2012-07-27',3007,5001),
19     (70008,5760.00,'2012-09-10',3002,5001),
20     (70010,1983.43,'2012-10-10',3004,5006),
21     (70003,2480.40,'2012-10-10',3009,5003),
22     (70012,250.45,'2012-06-27',3008,5002),
23     (70011,75.29,'2012-08-17',3003,5007),
24     (70013,3045.60,'2012-04-25',3002,5001);

```

Output

Action Output

#	Time	Action	Message
✓ 1	13:11:32	CREATE DATABASE orders_management_db	1 row(s) affected
✓ 2	13:11:34	USE orders_management_db	0 row(s) affected
✓ 3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cus...	0 row(s) affected
✓ 4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,5005), ...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0

Step 2 : WHERE

ORDERS TABLE

```

20     (70010,1983.43,'2012-10-10',3004,5006),
21     (70003,2480.40,'2012-10-10',3009,5003),
22     (70012,250.45,'2012-06-27',3008,5002),
23     (70011,75.29,'2012-08-17',3003,5007),
24     (70013,3045.60,'2012-04-25',3002,5001);
25     -- Step 3: WHERE Clause
26     • SELECT *
27     FROM orders
28     WHERE purch_amt > 2000;

```

Result Grid

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
✖	NULL	NULL	NULL	NULL	NULL

orders 1

Output

Action Output

#	Time	Action	Message
✓ 1	13:11:32	CREATE DATABASE orders_management_db	1 row(s) affected
✓ 2	13:11:34	USE orders_management_db	0 row(s) affected
✓ 3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cu...	0 row(s) affected
✓ 4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,5005)...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned

```
ORDERS TABLE x
Limit to 1000 rows
24 (70013,3045.60,'2012-04-25',3002,5001);
25 -- Step 3: WHERE Clause
26 • SELECT *
27 FROM orders
28 WHERE purch_amt > 2000;
29 -- Step 4: WHERE Clause with Date
30 • SELECT *
31 FROM orders
32 WHERE ord_date = '2012-09-10';
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: I

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
•	NULL	NULL	NULL	NULL	NULL

orders 2 x

Output

Action Output

#	Time	Action	Message
✓ 1	13:11:32	CREATE DATABASE orders_management_db	1 row(s) affected
✓ 2	13:11:34	USE orders_management_db	0 row(s) affected
✓ 3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, cu...	0 row(s) affected
✓ 4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,5005)...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
✓ 6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned

```
ORDERS TABLE x
Limit to 1000 rows
28 WHERE purch_amt > 2000;
29 -- Step 4: WHERE Clause with Date
30 • SELECT *
31 FROM orders
32 WHERE ord_date = '2012-09-10';
33 -- Step 5: WHERE Clause with Salesman
34 • SELECT *
35 FROM orders
36 WHERE salesman_id = 5001;
```

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: I

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70002	65.26	2012-10-05	3002	5001
	70005	2400.60	2012-07-27	3007	5001
	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
•	NULL	NULL	NULL	NULL	NULL

orders 3 x

Output

Action Output

#	Time	Action	Message
✓ 2	13:11:34	USE orders_management_db	0 row(s) affected
✓ 3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ...	0 row(s) affected
✓ 4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
✓ 5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
✓ 6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
✓ 7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned

Step 3 : ORDER BY

ORDERS TABLE

Limit to 1000 rows

```
32 WHERE ord_date = '2012-09-10';
33 -- Step 5: WHERE Clause with Salesman
34 • SELECT *
35 FROM orders
36 WHERE salesman_id = 5001;
37 -- Step 6: ORDER BY Purchase Amount
38 • SELECT *
39 FROM orders
40 ORDER BY purch_amt DESC;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70008	5760.00	2012-09-10	3002	5001
	70013	3045.60	2012-04-25	3002	5001
	70003	2480.40	2012-10-10	3009	5003
	70005	2400.60	2012-07-27	3007	5001
	70010	1983.43	2012-10-10	3004	5006
	70007	948.50	2012-09-10	3005	5002
	70009	270.65	2012-09-10	3001	5005
	70012	250.45	2012-06-27	3008	5002
	70001	150.50	2012-10-05	3005	5002

orders 4

Output

Action Output

#	Time	Action	Message
3	13:11:37	CREATE TABLE orders (ord_no INT PRIMARY KEY, purch_amt DECIMAL(10,2), ord_date DATE, ...	0 row(s) affected
4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned

ORDERS TABLE

Limit to 1000 rows

```
36 WHERE salesman_id = 5001;
37 -- Step 6: ORDER BY Purchase Amount
38 • SELECT *
39 FROM orders
40 ORDER BY purch_amt DESC;
41 -- Step 7: ORDER BY Order Date
42 • SELECT *
43 FROM orders
44 ORDER BY ord_date ASC;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	ord_no	purch_amt	ord_date	customer_id	salesman_id
▶	70013	3045.60	2012-04-25	3002	5001
	70012	250.45	2012-06-27	3008	5002
	70005	2400.60	2012-07-27	3007	5001
	70004	110.50	2012-08-17	3009	5003
	70011	75.29	2012-08-17	3003	5007
	70007	948.50	2012-09-10	3005	5002
	70008	5760.00	2012-09-10	3002	5001
	70009	270.65	2012-09-10	3001	5005
	70001	150.50	2012-10-05	3005	5002

orders 5

Output

Action Output

#	Time	Action	Message
4	13:12:11	INSERT INTO orders VALUES (70001,150.50,'2012-10-05',3005,5002), (70009,270.65,'2012-09-10',3001,500...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0
5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned

Step 4 : AGGREGATE FUNCTIONS

```
ORDERS TABLE x
39 FROM orders
40 ORDER BY purch_amt DESC;
41 -- Step 7: ORDER BY Order Date
42 • SELECT *
43 FROM orders
44 ORDER BY ord_date ASC;
45 -- Step 8: SUM()
46 • SELECT SUM(purch_amt) AS total_revenue
47 FROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

total_revenue
17541.18

Result 6 x

Output

Action Output

#	Time	Action	Message
✓ 5	13:12:38	SELECT * FROM orders WHERE purch_amt > 2000 LIMIT 0, 1000	4 row(s) returned
✓ 6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
✓ 7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned

ORDERS TABLE x

```
42 • SELECT *
43 FROM orders
44 ORDER BY ord_date ASC;
45 -- Step 8: SUM()
46 • SELECT SUM(purch_amt) AS total_revenue
47 FROM orders;
48 -- Step 9: AVG()
49 • SELECT AVG(purch_amt) AS average_order
50 FROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

average_order
1461.765000

Result 7 x

Output

Action Output

#	Time	Action	Message
✓ 6	13:13:12	SELECT * FROM orders WHERE ord_date = '2012-09-10' LIMIT 0, 1000	3 row(s) returned
✓ 7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
✓ 8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
✓ 9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned

ORDERS TABLE

Limit to 1000 rows

```

45  -- Step 8: SUM()
46  • SELECT SUM(purch_amt) AS total_revenue
47    FROM orders;
48  -- Step 9: AVG()
49  • SELECT AVG(purch_amt) AS average_order
50    FROM orders;
51  -- Step 10: MAX()
52  • SELECT MAX(purch_amt) AS highest_order
53    FROM orders;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

highest_order
5760.00

Result 8

Output

Action Output

#	Time	Action	Message
7	13:13:36	SELECT * FROM orders WHERE salesman_id = 5001 LIMIT 0, 1000	4 row(s) returned
8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned

ORDERS TABLE

Limit to 1000 rows

```

48  -- Step 9: AVG()
49  • SELECT AVG(purch_amt) AS average_order
50    FROM orders;
51  -- Step 10: MAX()
52  • SELECT MAX(purch_amt) AS highest_order
53    FROM orders;
54  -- Step 11: MIN()
55  • SELECT MIN(purch_amt) AS lowest_order
56    FROM orders;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

lowest_order
65.26

Result 9

Output

Action Output

#	Time	Action	Message
8	13:15:53	SELECT * FROM orders ORDER BY purch_amt DESC LIMIT 0, 1000	12 row(s) returned
9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned

ORDERS TABLE

```

51  -- Step 10: MAX()
52  • SELECT MAX(purch_amt) AS highest_order
53    FROM orders;
54  -- Step 11: MIN()
55  • SELECT MIN(purch_amt) AS lowest_order
56    FROM orders;
57  -- Step 12: COUNT()
58  • SELECT COUNT(*) AS total_orders
59    FROM orders;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: [IA](#)

	total_orders
▶	12

Result 10 ×

Output

Action Output

#	Time	Action	Message
✓ 9	13:16:24	SELECT * FROM orders ORDER BY ord_date ASC LIMIT 0, 1000	12 row(s) returned
✓ 10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned

Step 5 : GROUP BY

ORDERS TABLE

```

56  FROM orders;
57  -- Step 12: COUNT()
58  • SELECT COUNT(*) AS total_orders
59    FROM orders;
60  -- Step 13: GROUP BY Salesman
61  • SELECT salesman_id,
62          SUM(purch_amt) AS total_sales
63    FROM orders
64    GROUP BY salesman_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: [IA](#)

	salesman_id	total_sales
▶	5002	1349.45
	5001	11271.46
	5003	2590.90
	5005	270.65
	5006	1983.43
	5007	75.29

Result 11 ×

Output

Action Output

#	Time	Action	Message
✓ 10	13:16:48	SELECT SUM(purch_amt) AS total_revenue FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned

ORDERS TABLE

```

61 • SELECT salesman_id,
62       SUM(purch_amt) AS total_sales
63 FROM orders
64 GROUP BY salesman_id;
65 -- Step 14: GROUP BY Customer
66 • SELECT customer_id,
67       SUM(purch_amt) AS total_purchase
68 FROM orders
69 GROUP BY customer_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	total_purchase
3005	1099.00
3002	8870.86
3009	2590.90
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Result 12 x

Output

Action Output

#	Time	Action	Message
✓ 11	13:17:11	SELECT AVG(purch_amt) AS average_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned

ORDERS TABLE

```

66 • SELECT customer_id,
67       SUM(purch_amt) AS total_purchase
68 FROM orders
69 GROUP BY customer_id;
70 -- Step 15: GROUP BY + MAX()
71 • SELECT customer_id,
72       MAX(purch_amt) AS highest_purchase
73 FROM orders
74 GROUP BY customer_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_id	highest_purchase
3005	948.50
3002	5760.00
3009	2480.40
3007	2400.60
3001	270.65
3004	1983.43
3003	75.29
3008	250.45

Result 13 x

Output

Action Output

#	Time	Action	Message
✓ 12	13:17:38	SELECT MAX(purch_amt) AS highest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIMIT...	8 row(s) returned

Step 6 : GROUP BY +HAVING

ORDERS TABLE

```
72      MAX(purch_amt) AS highest_purchase
73      FROM orders
74      GROUP BY customer_id;
75      -- Step 16: GROUP BY + HAVING
76 •    SELECT salesman_id,
77           SUM(purch_amt) AS total_sales
78      FROM orders
79      GROUP BY salesman_id
80      HAVING SUM(purch_amt) > 3000;
```

Result Grid

	salesman_id	total_sales
▶	5001	11271.46

Result 14

Output

Action Output

#	Time	Action	Message
✓ 13	13:18:00	SELECT MIN(purch_amt) AS lowest_order FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIMIT...	8 row(s) returned
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned

ORDERS TABLE

```
78      FROM orders
79      GROUP BY salesman_id
80      HAVING SUM(purch_amt) > 3000;
81      -- Step 17: GROUP BY + HAVING
82 •    SELECT customer_id,
83           SUM(purch_amt) AS total_purchase
84      FROM orders
85      GROUP BY customer_id
86      HAVING SUM(purch_amt) > 2500;
```

Result Grid

	customer_id	total_purchase
▶	3002	8870.86
	3009	2590.90

Result 15

Output

Action Output

#	Time	Action	Message
✓ 14	13:18:26	SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000	1 row(s) returned
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIMIT...	8 row(s) returned
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	2 row(s) returned

ORDERS TABLE

```

84 FROM orders
85 GROUP BY customer_id
86 HAVING SUM(purch_amt) > 2500;
87 -- Step 18: GROUP BY + HAVING
88 • SELECT customer_id,
89     COUNT(*) AS total_orders
90 FROM orders
91 GROUP BY customer_id
92 HAVING COUNT(*) > 1;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_orders
▶	3005	2
	3002	3
	3009	2

Result 16 ×

Output

Action Output

#	Time	Action	Message
✓ 15	13:18:46	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id LIMIT 0, 1...	6 row(s) returned
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIM...	8 row(s) returned
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	2 row(s) returned
✓ 20	13:20:58	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUNT...	3 row(s) returned

Step 7: GROUP BY +HAVING+ORDER BY

ORDERS TABLE

```

91 GROUP BY customer_id
92 HAVING COUNT(*) > 1;
93 -- Step 19: GROUP BY + HAVING + ORDER BY
94 • SELECT customer_id,
95     SUM(purch_amt) AS total_purchase
96 FROM orders
97 GROUP BY customer_id
98 HAVING SUM(purch_amt) > 1000
99 ORDER BY total_purchase DESC;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	total_purchase
▶	3002	8870.86
	3009	2590.90
	3007	2400.60
	3004	1983.43
	3005	1099.00

Result 17 ×

Output

Action Output

#	Time	Action	Message
✓ 16	13:19:08	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id LIMIT ...	8 row(s) returned
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIM...	8 row(s) returned
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	2 row(s) returned
✓ 20	13:20:58	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUNT...	3 row(s) returned
✓ 21	13:21:22	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	5 row(s) returned

Step 8 : BETWEEN + HAVING

ORDERS TABLE

```
97 GROUP BY customer_id
98 HAVING SUM(purch_amt) > 1000
99 ORDER BY total_purchase DESC;
100 -- Step 20: BETWEEN + HAVING
101 • SELECT customer_id,
102       MAX(purch_amt) AS max_purchase
103 FROM orders
104 GROUP BY customer_id
105 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
```

Result Grid

	customer_id	max_purchase
▶	3002	5760.00
	3009	2480.40
	3007	2400.60

Result 18

Output

Action Output

#	Time	Action	Message
✓ 17	13:19:31	SELECT customer_id, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY customer_id LIMIT...	8 row(s) returned
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	2 row(s) returned
✓ 20	13:20:58	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUNT...	3 row(s) returned
✓ 21	13:21:22	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	5 row(s) returned
✓ 22	13:21:52	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY customer_id HAVIN...	3 row(s) returned

Step 9 : COUNT+HAVING

ORDERS TABLE

```
103 FROM orders
104 GROUP BY customer_id
105 HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
106 -- Step 21: COUNT + HAVING
107 • SELECT salesman_id,
108       COUNT(*) AS total_orders
109 FROM orders
110 GROUP BY salesman_id
111 HAVING COUNT(*) >= 2;
```

Result Grid

	salesman_id	total_orders
▶	5002	3
	5001	4
	5003	2

Result 19

Output

Action Output

#	Time	Action	Message
✓ 18	13:19:59	SELECT salesman_id, SUM(purch_amt) AS total_sales FROM orders GROUP BY salesman_id HAVING S...	1 row(s) returned
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	2 row(s) returned
✓ 20	13:20:58	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUNT...	3 row(s) returned
✓ 21	13:21:22	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVIN...	5 row(s) returned
✓ 22	13:21:52	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY customer_id HAVIN...	3 row(s) returned
✓ 23	13:26:41	SELECT salesman_id, COUNT(*) AS total_orders FROM orders GROUP BY salesman_id HAVING COUN...	3 row(s) returned

Step 10 : MAX+GROUP BY + HAVING

ORDERS TABLE x

Limit to 1000 rows

```
109 FROM orders
110 GROUP BY salesman_id
111 HAVING COUNT(*) >= 2;
112 -- Step 22: MAX + GROUP BY + HAVING
113 • SELECT ord_date,
114        MAX(purch_amt) AS highest_purchase
115 FROM orders
116 GROUP BY ord_date
117 HAVING MAX(purch_amt) > 2000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Contents: |

	ord_date	highest_purchase
▶	2012-10-10	2480.40
	2012-07-27	2400.60
	2012-09-10	5760.00
	2012-04-25	3045.60

Result 20 x

Output

Action Output

#	Time	Action	Message
✓ 19	13:20:37	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVING COUNT(*) >= 2;	2 row(s) returned
✓ 20	13:20:58	SELECT customer_id, COUNT(*) AS total_orders FROM orders GROUP BY customer_id HAVING COUNT(*) >= 2;	3 row(s) returned
✓ 21	13:21:22	SELECT customer_id, SUM(purch_amt) AS total_purchase FROM orders GROUP BY customer_id HAVING COUNT(*) >= 2;	5 row(s) returned
✓ 22	13:21:52	SELECT customer_id, MAX(purch_amt) AS max_purchase FROM orders GROUP BY customer_id HAVING COUNT(*) >= 2;	3 row(s) returned
✓ 23	13:26:41	SELECT salesman_id, COUNT(*) AS total_orders FROM orders GROUP BY salesman_id HAVING COUNT(*) >= 2;	3 row(s) returned
✓ 24	13:28:15	SELECT ord_date, MAX(purch_amt) AS highest_purchase FROM orders GROUP BY ord_date HAVING MAX(purch_amt) > 2000;	4 row(s) returned